



<https://algs4.cs.princeton.edu>

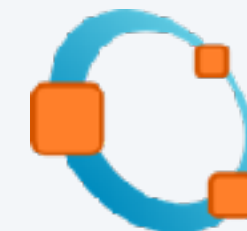
2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*

Two classic sorting algorithms: mergesort and quicksort

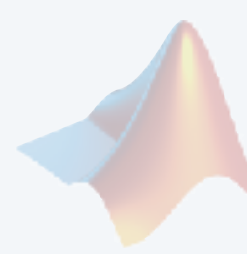
Critical components in our computational infrastructure.

Mergesort. [this lecture]



...

Quicksort. [next lecture]



...



<https://algs4.cs.princeton.edu>

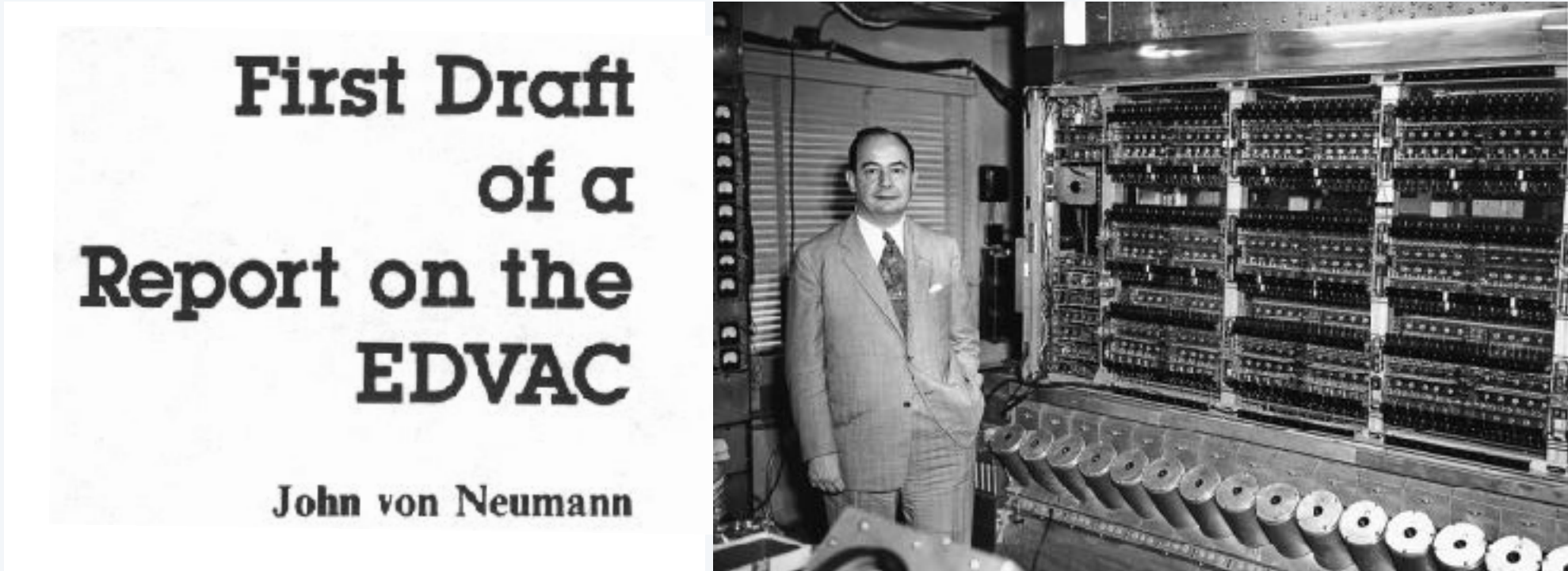
2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*

Mergesort overview

Basic plan.

- Divide array into two halves.
- Recursively sort left half.
- Recursively sort right half.
- **Merge** two sorted halves.

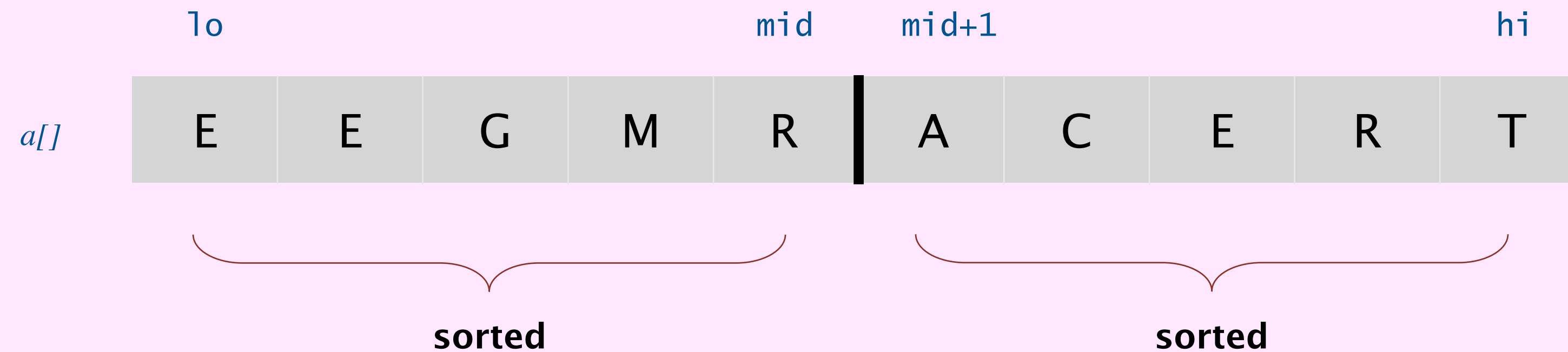


input	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E	合并排序，递归二分
sort left half	E	E	G	M	O	R	R	S	T	E	X	A	M	P	L	E	
sort right half	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X	
merge results	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X	

Abstract in-place merge demo



Goal. Given two sorted subarrays $a[lo]$ to $a[mid]$ and $a[mid+1]$ to $a[hi]$, replace with sorted subarray $a[lo]$ to $a[hi]$.



Merging: Java implementation

```
private static void merge(Comparable[] a, Comparable[] aux, int lo, int mid, int hi) {
```

```
    for (int k = lo; k <= hi; k++)  
        aux[k] = a[k];
```

copy 理解：访问了2次数组，aux和a各一次
先把原数组 a 中 [lo, hi] 区间的元素完整复制到辅助数组 aux 中。
目的：避免在合并时直接覆盖原数组的有效数据。

```
    int i = lo, j = mid+1;  
    for (int k = lo; k <= hi; k++) {  
        if (i > mid) a[k] = aux[j++];  
        else if (j > hi) a[k] = aux[i++];  
        else if (less(aux[j], aux[i])) a[k] = aux[j++];  
        else a[k] = aux[i++];  
    }
```

merge 区别元素比较和下标比较

最坏情况：比较n次！！！！（左边的头，一直比到右边的尾，然后左边的其他，和右边的尾部比较）
并不会出现任意两两都会比较的情况！

```
}
```

用双指针分别遍历两个有序子数组，从辅助数组 aux 中按大小依次取出元素，写回原数组 a。

如果左子数组已遍历完 (i > mid)：直接取右子数组当前元素 aux[j++]
如果右子数组已遍历完 (j > hi)：直接取左子数组当前元素 aux[i++]
否则比较 aux[i] 和 aux[j]：
取较小的那个放入 a[k]，并将对应指针后移





How many calls does `merge()` make to `less()` when merging two sorted subarrays, each of length $n / 2$, into a sorted array of length n ?

A. $\sim \frac{1}{4} n$ to $\sim \frac{1}{2} n$

B. $\sim \frac{1}{2} n$

C. $\sim \frac{1}{2} n$ to $\sim n$

D. $\sim n$

merging two sorted arrays, each of length $n/2$



Mergesort: Java implementation

```
public class Merge {  
    private static void merge(...) {  
        /* as before */  
    }  
}
```

```
private static void sort(Comparable[] a, Comparable[] aux, int lo, int hi) {  
    if (hi <= lo) return;  
    int mid = lo + (hi - lo) / 2;  
    sort(a, aux, lo, mid);  
    sort(a, aux, mid+1, hi);  
    merge(a, aux, lo, mid, hi);  
}
```

一般是向下取整



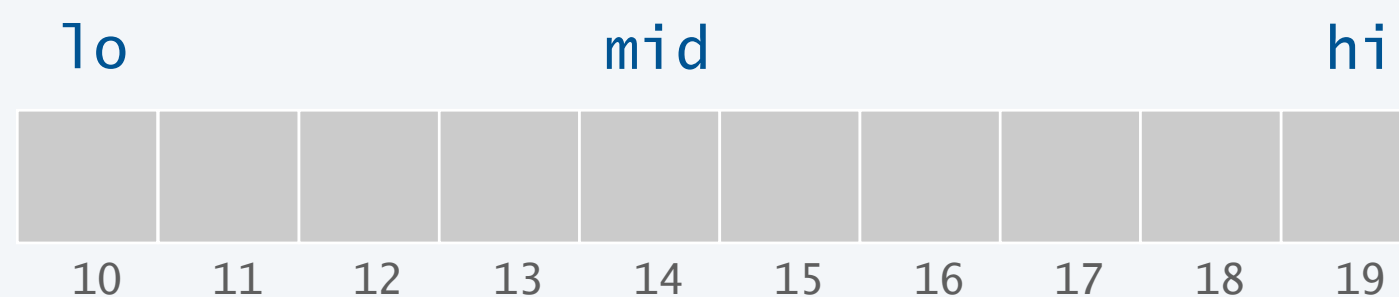
千万注意，严格按照算法来看的话，长度为奇数的会被划分成长+短：

比如3：lo=0, hi=2, mid=0+1=1，也就是 (0, 1) (2, 2) 这两个

5：(0, 4) 拆成 (0, 2) (2, 4)

*avoid allocating arrays
within recursive function calls*

```
public static void sort(Comparable[] a) {  
    Comparable[] aux = new Comparable[a.length];  
    sort(a, aux, 0, a.length - 1);  
}
```



Mergesort: trace

					a[]																
					0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
					M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E	
merge(a, aux, ^{lo} 0, 0, ^{hi} 1)					E	M	R	G	E	S	O	R	T	E	X	A	M	P	L	E	← result after recursive call
merge(a, aux, 2, 2, 3)					E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E	
merge(a, aux, 0, 1, 3)					E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E	
merge(a, aux, 4, 4, 5)					E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E	
merge(a, aux, 6, 6, 7)					E	G	M	R	E	S	O	R	T	E	X	A	M	P	L	E	
merge(a, aux, 4, 5, 7)					E	G	M	R	E	O	R	S	T	E	X	A	M	P	L	E	
merge(a, aux, 0, 3, 7)					E	E	G	M	O	R	R	S	T	E	X	A	M	P	L	E	
merge(a, aux, 8, 8, 9)					E	E	G	M	O	R	R	S	E	T	X	A	M	P	L	E	
merge(a, aux, 10, 10, 11)					E	E	G	M	O	R	R	S	E	T	A	X	M	P	L	E	
merge(a, aux, 8, 9, 11)					E	E	G	M	O	R	R	S	A	E	T	X	M	P	L	E	
merge(a, aux, 12, 12, 13)					E	E	G	M	O	R	R	S	A	E	T	X	M	P	L	E	
merge(a, aux, 14, 14, 15)					E	E	G	M	O	R	R	S	A	E	T	X	M	P	E	L	
merge(a, aux, 12, 13, 15)					E	E	G	M	O	R	R	S	A	E	T	X	E	L	M	P	
merge(a, aux, 8, 11, 15)					E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X	
merge(a, aux, 0, 7, 15)					A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X	



Which subarray lengths will arise when mergesorting an array of length 12?

A. { 1, 2, 3, 4, 6, 8, 12 }

B. { 1, 2, 3, 6, 12 }

C. { 1, 2, 4, 8, 12 }

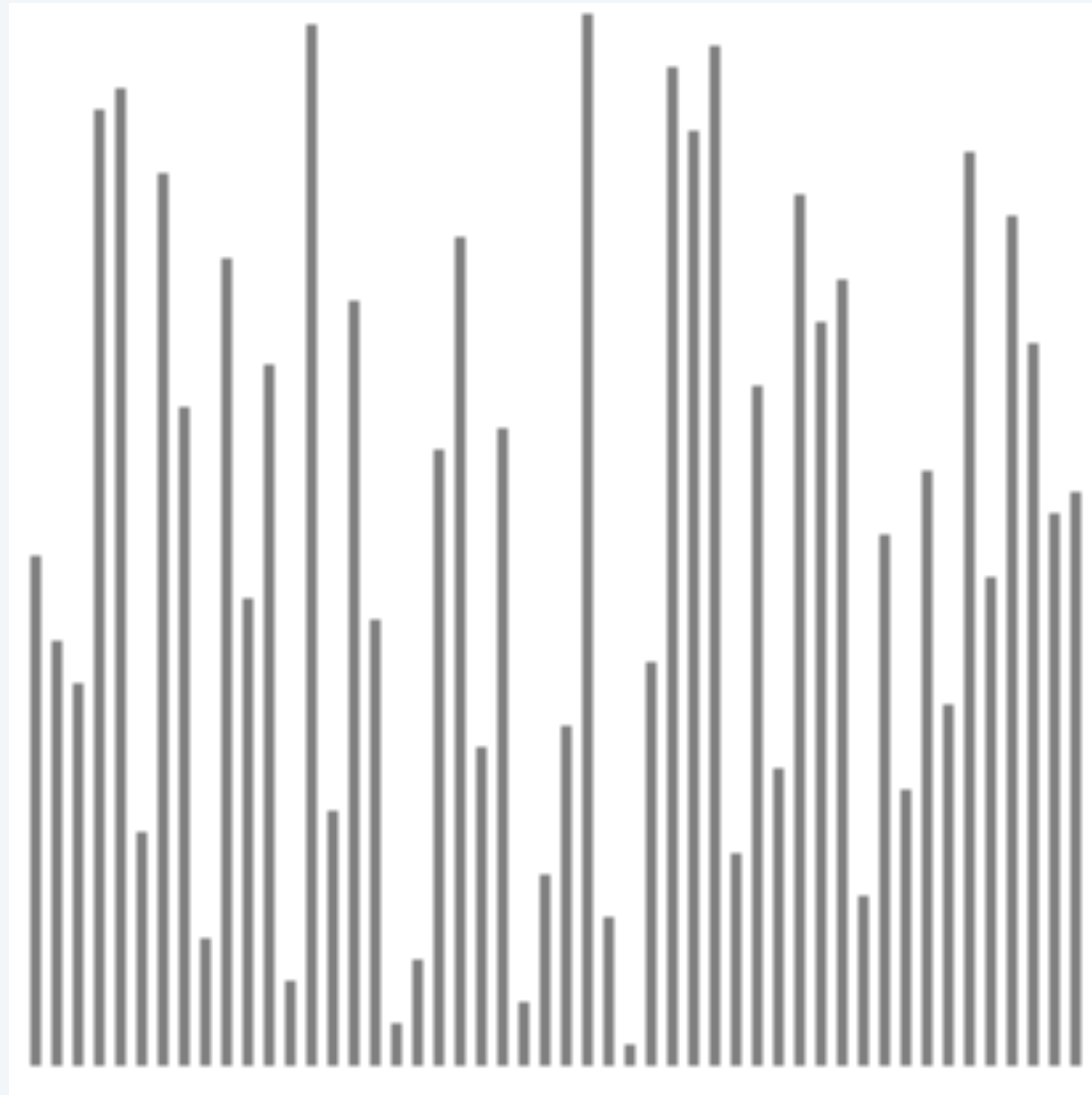
D. { 1, 3, 6, 9, 12 }

题意：原始长度是12，那么在合并排序的过程中，哪些子数组的长度会出现？

12-6-3，注意3会被分成2和1
所以选择B

Mergesort: animation

50 random items

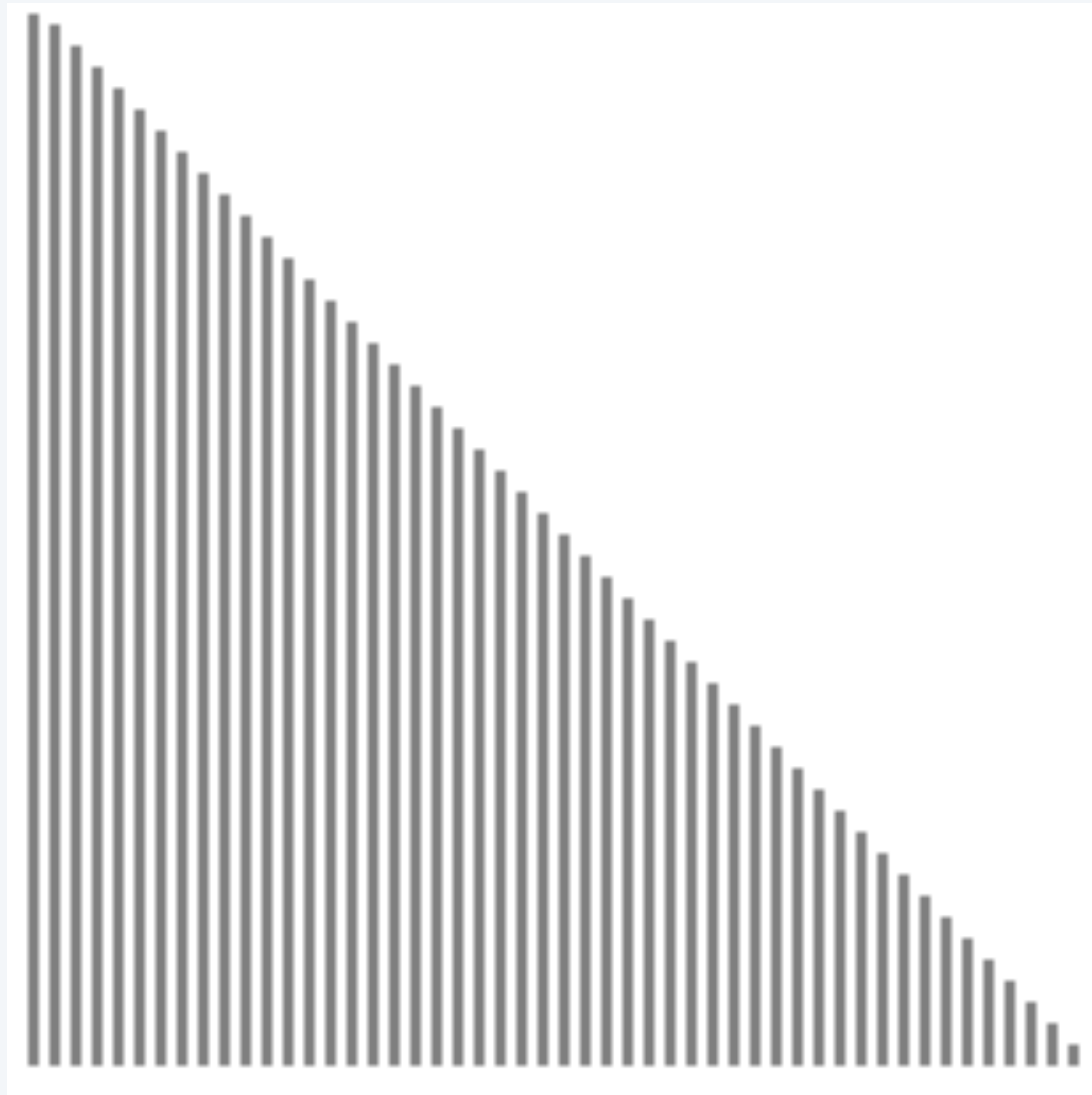


<https://www.toptal.com/developers/sorting-algorithms/merge-sort>

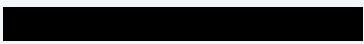
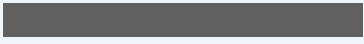
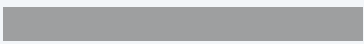
- ▲ algorithm position
- in order
- current subarray
- not in order

Mergesort: animation

50 reverse-sorted items



<https://www.toptal.com/developers/sorting-algorithms/merge-sort>

-  algorithm position
-  in order
-  current subarray
-  not in order

Mergesort: empirical analysis

Running time estimates:

- Laptop executes 10^8 compares/second.
- Supercomputer executes 10^{12} compares/second.

	insertion sort (n^2)			mergesort ($n \log n$)		
computer	thousand	million	billion	thousand	million	billion
home	<i>instant</i>	2.8 hours	317 years	<i>instant</i>	1 second	18 min
super	<i>instant</i>	1 second	1 week	<i>instant</i>	<i>instant</i>	<i>instant</i>

Bottom line. Good algorithms are better than supercomputers.

Mergesort analysis: number of compares

Proposition. Mergesort uses $\leq n \log_2 n$ compares to sort any array of length n .

Pf sketch. The number of compares $C(n)$ to mergesort any array of length n satisfies the **recurrence**:

$$C(n) \leq C(\lceil n/2 \rceil) + C(\lfloor n/2 \rfloor) + n - 1 \quad \text{for } n > 1, \text{ with } C(1) = 0.$$

$\uparrow$ $\uparrow$ $\uparrow$
sort *sort* *merge*
left half *right half*

*proposition holds even when n is not a power of 2
(but analysis cleaner in this case)*

For simplicity. Assume n is a power of 2 and solve this recurrence:

假设n是2的幂次

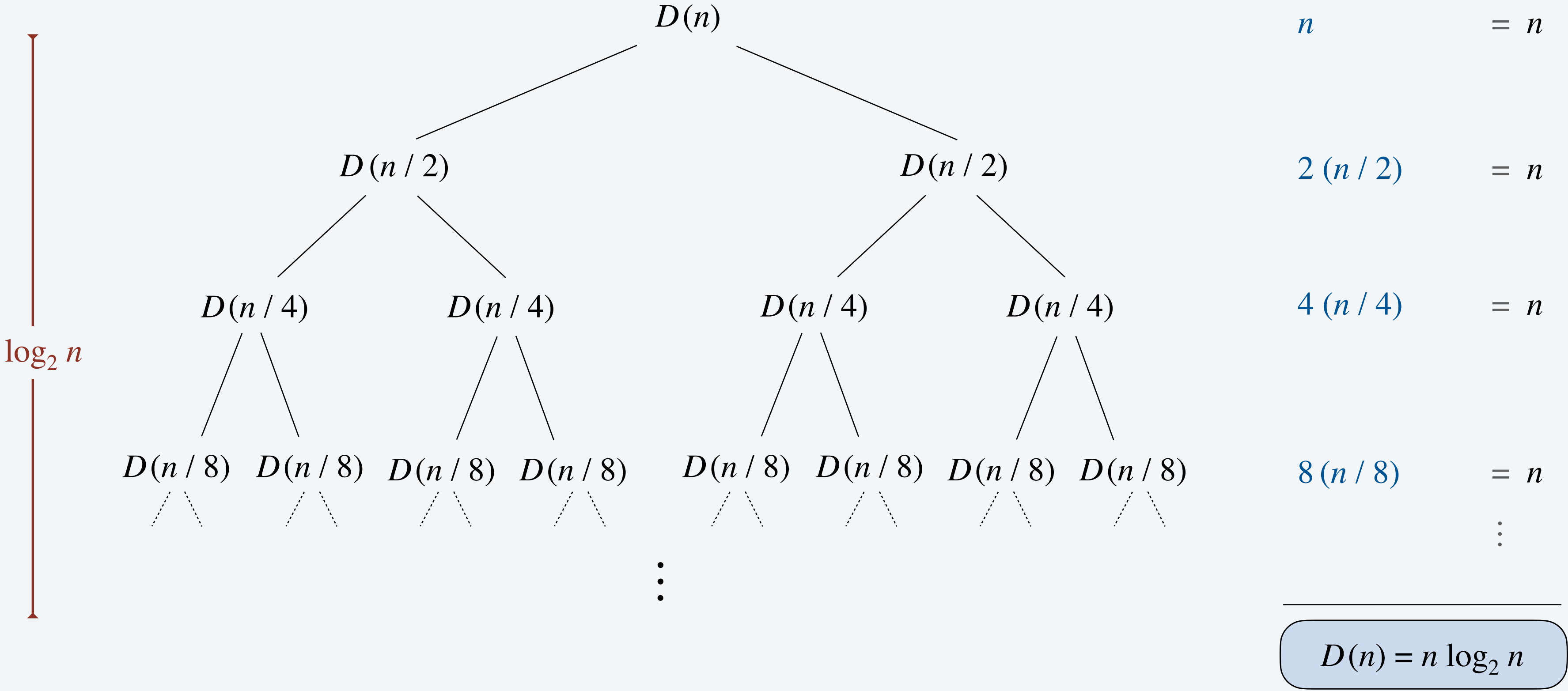
$$D(n) = 2 D(n/2) + n, \text{ for } n > 1, \text{ with } D(1) = 0.$$

Divide-and-conquer recurrence

$D(n)$ 是对 n 个数进行排序时候所需的元素比较次数
 $D(1) = 0$!

Proposition. If $D(n)$ satisfies $D(n) = 2 D(n / 2) + n$ for $n > 1$, with $D(1) = 0$, then $D(n) = n \log_2 n$.

Pf by picture. [assuming n is a power of 2] n 是在递归后的两个子数组的时间开销



Mergesort analysis: number of array accesses

Proposition. Mergesort makes $\Theta(n \log n)$ array accesses.

Pf sketch. The number of array accesses $A(n)$ satisfies the recurrence:

$$A(n) = A(\lceil n/2 \rceil) + A(\lfloor n/2 \rfloor) + \Theta(n) \text{ for } n > 1, \text{ with } A(1) = 0.$$

Key point. Any algorithm with the following structure takes $\Theta(n \log n)$ time:

```
public static void f(int n) {  
    if (n == 0) return;  
    f(n/2);  
    f(n/2);  
    linear(n);  
}
```

← solve two problems of half the size

← do $\Theta(n)$ work

*what happens if we call
 $f(n/3)$ three times?*

Famous examples. FFT, closest pair, hidden-line removal, Kendall-tau distance, ...

Mergesort analysis: memory

Proposition. Mergesort uses $\Theta(n)$ extra space.

Pf. The length of the `aux[]` array is n , to handle the last merge. aux是辅助数组

two sorted subarrays

A C D G H I M N U V

B E F J O P Q R S T

merged result

A B C D E F G H I J M N O P Q R S T U V

Def. A sorting algorithm is **in-place** if it uses $\Theta(\log n)$ extra space (or less).

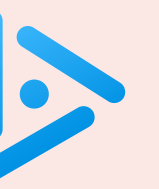
Ex. Insertion sort and selection sort.

Challenge 1 (not hard). Get by with an `aux[]` array of length $\sim \frac{1}{2} n$ (instead of n).

Challenge 2 (very hard). In-place merge. [Kronrod 1969]

essentially negligible





Consider the following **modified** version of mergesort.

How much total memory is allocated over all recursive calls?

- A. $\Theta(n)$
- B. $\Theta(n \log n)$
- C. $\Theta(n^2)$
- D. $\Theta(2^n)$

```
private static void sort(Comparable[] a, int lo, int hi) {  
    if (hi <= lo) return;  
    int mid = lo + (hi - lo) / 2;  
    int n = hi - lo + 1;  
    Comparable[] aux = new Comparable[n];  
    sort(a, lo, mid);  
    sort(a, mid+1, hi);  
    merge(a, aux, lo, mid, hi);  
}
```

每一层的每一次递归，都需要初始化新的存储空间
每一层一共需要n的新空间
一共有logn层

Mergesort: practical improvement

Use insertion sort for small subarrays.

- Mergesort has too much overhead for tiny subarrays.
- Cutoff to insertion sort for ≈ 10 items.

```
private static void sort(...) {
```

```
    if (hi <= lo + CUTOFF - 1) {  
        Insertion.sort(a, lo, hi);  
        return;  
    }
```

*makes mergesort
about 20% faster*

```
    int mid = lo + (hi - lo) / 2;  
    sort (a, aux, lo, mid);  
    sort (a, aux, mid+1, hi);  
    merge(a, aux, lo, mid, hi);  
}
```



<https://algs4.cs.princeton.edu>

2.2 MERGESORT

- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*

Bottom-up mergesort

Basic plan.

- Pass through array, merging subarrays of length 1.
- Repeat for subarrays of length 2, 4, 8, ...

	a[i]															
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
sz = 1	M	E	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 0, 0, 1)	E	M	R	G	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 2, 2, 3)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 4, 4, 5)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 6, 6, 7)	E	M	G	R	E	S	O	R	T	E	X	A	M	P	L	E
merge(a, aux, 8, 8, 9)	E	M	G	R	E	S	O	R	E	T	X	A	M	P	L	E
merge(a, aux, 10, 10, 11)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	L	E
merge(a, aux, 12, 12, 13)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	L	E
merge(a, aux, 14, 14, 15)	E	M	G	R	E	S	O	R	E	T	A	X	M	P	E	L
sz = 2	E	G	M	R	E	S	O	R	E	T	A	X	M	P	E	L
merge(a, aux, 4, 5, 7)	E	G	M	R	E	O	R	S	E	T	A	X	M	P	E	L
merge(a, aux, 8, 9, 11)	E	G	M	R	E	O	R	S	A	E	T	X	M	P	E	L
merge(a, aux, 12, 13, 15)	E	G	M	R	E	O	R	S	A	E	T	X	E	L	M	P
sz = 4	E	E	G	M	O	R	R	S	A	E	T	X	E	L	M	P
merge(a, aux, 8, 11, 15)	E	E	G	M	O	R	R	S	A	E	E	L	M	P	T	X
sz = 8	A	E	E	E	E	G	L	M	M	O	P	R	R	S	T	X

Bottom-up mergesort: Java implementation

自底向上 (Bottom-up) 【迭代】

```
public class MergeBU {  
  
    private static void merge(...) {  
        /* as before */  
    }  
  
    public static void sort(Comparable[] a) {  
        int n = a.length;  
        Comparable[] aux = new Comparable[n];  
        for (int sz = 1; sz < n; sz = sz+sz)   
            for (int lo = 0; lo < n-sz; lo += sz+sz)  
                merge(a, aux, lo, lo+sz-1, Math.min(lo+sz+sz-1, n-1));  
    }  
}
```

// 外层循环：控制子数组大小 sz，从1开始，每次翻倍 (1→2→4→8...)
// 内层循环：遍历所有待归并的子数组对，每次步进 2 *sz

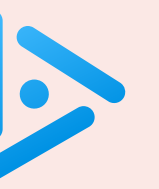
mid hi

这一步保证了自底向上的归并排序可以适合任何长度的数组！
这也是与自顶向下的核心区别之一

Proposition. At most $n \log_2 n$ compares; $\Theta(n)$ extra space.

Bottom line. Simple and non-recursive version of mergesort.

见下页



Which is faster in practice for $n = 2^{20}$, top-down mergesort or bottom-up mergesort?

A. Top-down (recursive) mergesort.

B. Bottom-up (non-recursive) mergesort.

C. No difference. 时间复杂度：两种都是每层最多 n 次比较，一共 $\log n$ 层，所以都是 $n \log n$

D. *I don't know.* 空间复杂度：
自底向上归并排序需要一个辅助数组来暂存归并过程中的数据，辅助数组的长度和原数组一致（ n ），因此：
空间复杂度： $O(n)$ （和递归版一样，都是「非原地排序」）；
但是注意：递归版会额外有 $O(\log n)$ 的递归栈空间，但自底向上是迭代实现，没有栈开销，这是两者的唯一区别（但不影响「渐近复杂度」）。

在实践当中，栈的开销会影响速度，因此迭代这个方式会更高效（尤其是当数据量增大的时候）

但也有另一个角度（缓存命中开销）：

递归会先处理较小的连续子数组小块数据更容易留在 CPU cache 中局部性可能更好

Natural mergesort

Idea. Exploit pre-existing order by identifying naturally occurring runs.

input

1	5	10	16	3	4	23	9	13	2	7	8	12	14
---	---	----	----	---	---	----	---	----	---	---	---	----	----



first run

1	5	10	16	3	4	23	9	13	2	7	8	12	14
---	---	----	----	---	---	----	---	----	---	---	---	----	----

second run

1	5	10	16	3	4	23	9	13	2	7	8	12	14
---	---	----	----	---	---	----	---	----	---	---	---	----	----

merge two runs

1	3	4	5	10	16	23	9	13	2	7	8	12	14
---	---	---	---	----	----	----	---	----	---	---	---	----	----

Tradeoff. Fewer passes vs. extra compares per pass to identify runs.

Timsort (2002)

- Natural mergesort.
- Use binary insertion sort to make initial runs (if needed).
- A few more clever optimizations.

This describes an adaptive, stable, natural mergesort, modestly called timsort (hey, I earned it <wink>). It has supernatural performance on many kinds of partially ordered arrays (less than $\lg(n!)$ comparisons needed, and as few as $n-1$), yet as fast as Python's previous highly tuned samplesort hybrid on random arrays.

In a nutshell, the main routine marches over the array once, left to right, alternately identifying the next run, then merging it into the previous runs "intelligently". Everything else is complication for speed, and some hard-won measure of memory efficiency.

...



Tim Peters

Consequence. Only $\Theta(n)$ compares on many arrays with pre-existing order.

Widely used. Python, Java, GNU Octave, Android, ...



Envisage: Engineering Virtualized Services

Envisage About Envisage Follow Envisage Dissemination Log in 

Proving that Android's, Java's and Python's sorting algorithm is broken (and showing how to fix it)

🕒 February 24, 2015 📁 Envisage ✍️ Written by Stijn de Gouw. 💰 \$s

Tim Peters developed the **Timsort hybrid sorting algorithm** in 2002. It is a clever combination of ideas from merge sort and insertion sort, and designed to perform well on real world data. TimSort was first developed for Python, but later ported to Java (where it appears as `java.util.Collections.sort` and `java.util.Arrays.sort`) by **Joshua Bloch** (the designer of Java Collections who also pointed out that **most binary search algorithms were broken**). TimSort is today used as the default sorting algorithm for Android SDK, Sun's JDK and OpenJDK. Given the popularity of these platforms this means that the number of computers, cloud services and mobile phones that use TimSort for sorting is well into the billions.

Timsort bug (May 2018)



JDK / [JDK-8203864](#)

Execution error in Java's Timsort

Details

Type:	Bug
Status:	RESOLVED
Priority:	3 P3
Resolution:	Fixed
Affects Version/s:	None
Fix Version/s:	11
Component/s:	core-libs
Labels:	None
Subcomponent:	java.util.collections
Introduced In Version:	6
Resolved In Build:	b20

Description

Carine Pivoteau wrote:
While working on a proper complexity analysis of the algorithm, we realised that there was an error in the last paper reporting such a bug (<http://envisage-project.eu/wp-content/uploads/2015/02/sorting.pdf>). This implies that the correction implemented in the Java source code (changing Timsort stack size) is wrong and that it is still possible to make it break. This is explained in full details in our analysis: <https://arxiv.org/pdf/1805.08612.pdf>.
We understand that coming upon data that actually causes this error is very unlikely, but we thought you'd still like to know and do something about it. As the authors of the previous article advocated for, we strongly believe that you should consider modifying the algorithm as explained in their article (and as was done in Python) rather than trying to fix the stack size.

Sorting summary

	in-place?	stable?	best	average	worst	remarks
selection	✓		$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	n exchanges
insertion	✓	✓	n	$\frac{1}{4} n^2$	$\frac{1}{2} n^2$	use for small n or partially sorted
merge		✓	$\frac{1}{2} n \log_2 n$	$n \log_2 n$	$n \log_2 n$	$\Theta(n \log n)$ guarantee; stable
timsort		✓	n	$n \log_2 n$	$n \log_2 n$	improves mergesort when pre-existing order
?	✓	✓	n	$n \log_2 n$	$n \log_2 n$	holy sorting grail

number of compares to sort an array of n elements



<https://algs4.cs.princeton.edu>

2.2 MERGESORT

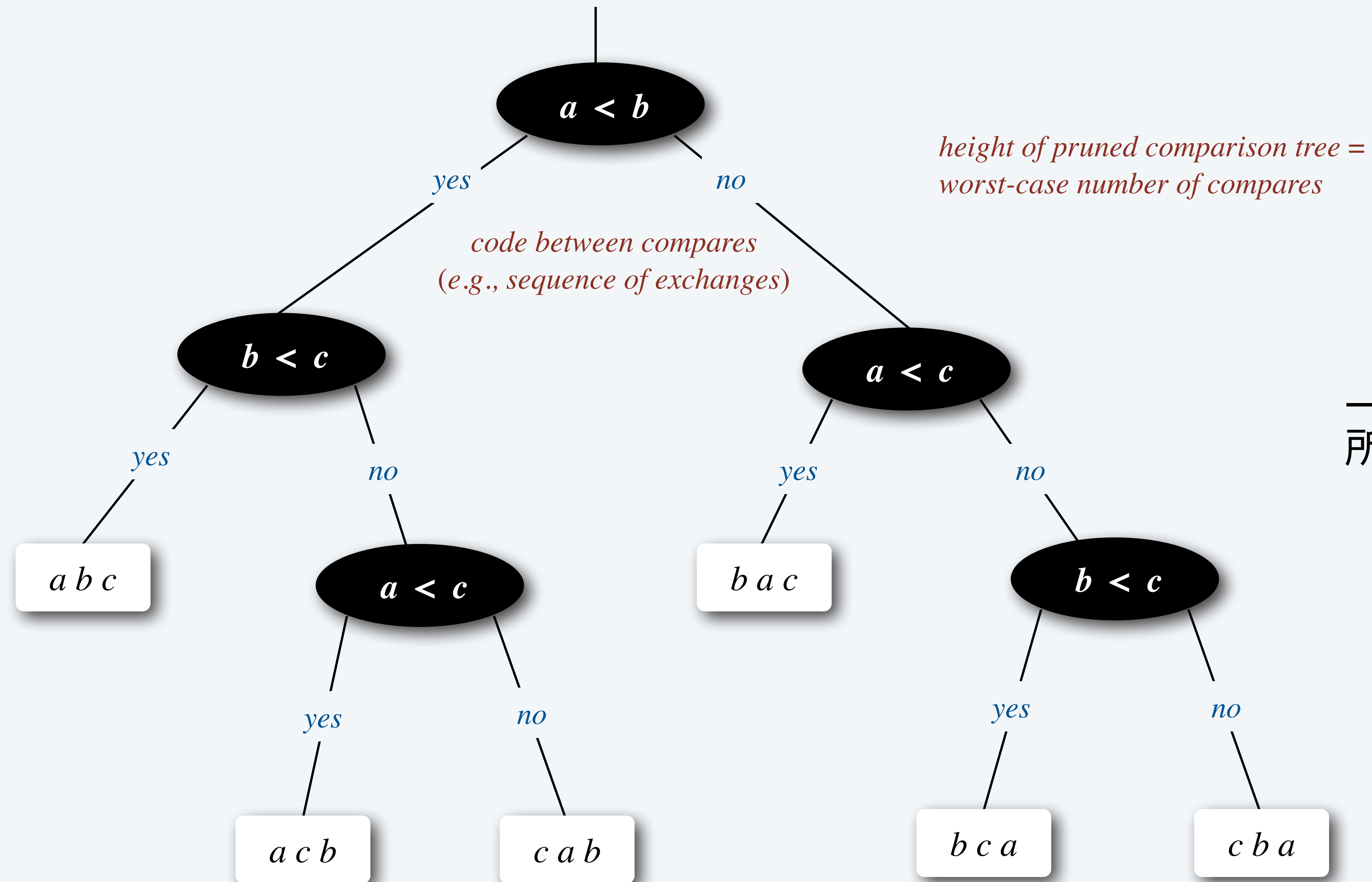
- *mergesort*
- *bottom-up mergesort*
- *sorting complexity*

Computational complexity

A framework to study efficiency of algorithms for solving a particular problem X .

term	description	example (X = sorting)
规定算法能做什么 model of computation	<i>specifies memory and primitive operations</i>	comparison tree ← <i>can gain knowledge about input only through pairwise compares (e.g., Java's Comparable framework)</i>
规定以什么作为开销分析对象 cost model	<i>primitive operation counts</i>	# compares
某个算法最大开销 upper bound	<i>cost guarantee provided by some algorithm for a problem</i>	$\sim n \log_2 n$ ← <i>from mergesort</i>
对某个问题，所有算法最小开销 lower bound	<i>proven limit on cost guarantee for all algorithms for a problem</i>	?
optimal algorithm	<i>algorithm with best possible cost guarantee for a problem</i>	?
开销是lower bound的算法	↑ <i>lower bound ~ upper bound</i>	

Comparison tree (for 3 distinct keys a, b, and c)



一个叶子节点代表一个排列的顺序
所以叶子节点的数量就是n!

one (and only one) reachable leaf corresponds to each possible ordering

Compare-based lower bound for sorting

Proposition. In the worst case, any compare-based sorting algorithm must make at least

$\log_2(n!) \sim n \log_2 n$ compares. 符号 $\sim$ 表示渐近等价, n 趋于无穷的时候比值为1

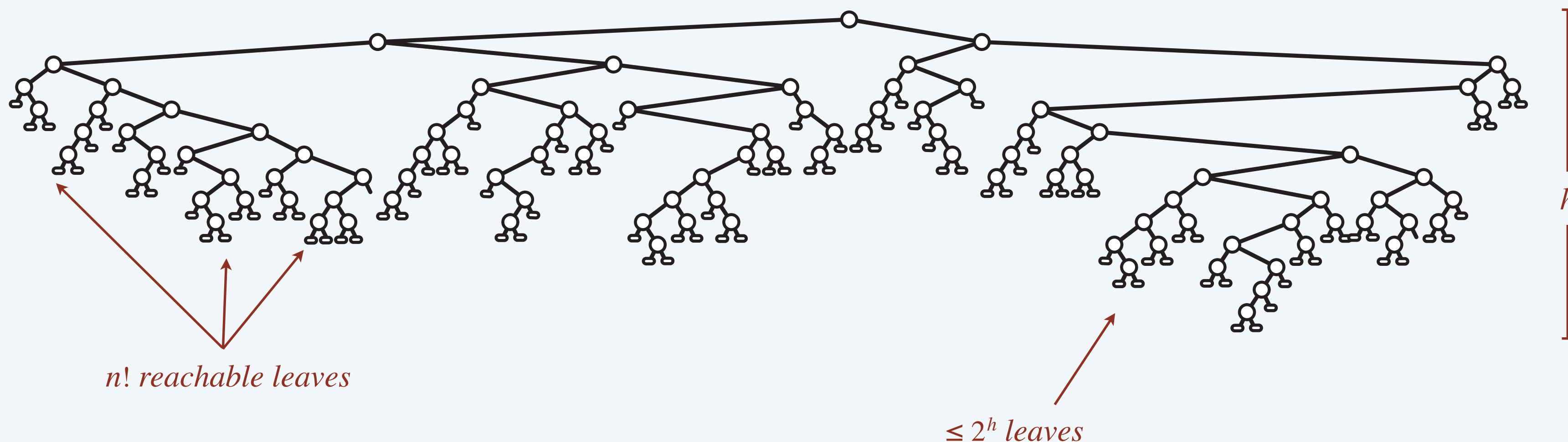
斯特林公式

Pf.

- Assume array consists of n distinct values a_1 through a_n . $n!$ 种排列: n 个不同元素的全排列有 $n!$ 种, 对应比较树的 $n!$ 个叶子节点。
- $n!$ different orderings $\Rightarrow n!$ reachable leaves.
- Worst-case number of compares = height h of pruned comparison tree.
- Binary tree of height h has $\leq 2^h$ leaves.

比较树: 把每次元素比较看作二叉树的一个节点, 左分支表示 “ $a < b$ ”, 右分支表示 “ $a > b$ ”, 叶子节点对应一个最终的排序结果。

高度为 h 的二叉树, 最多有 2^h 个叶子节点。



Compare-based lower bound for sorting

Proposition. In the worst case, any compare-based sorting algorithm must make at least $\log_2(n!) \sim n \log_2 n$ compares.

Pf.

- Assume array consists of n distinct values a_1 through a_n .
- $n!$ different orderings $\Rightarrow n!$ reachable leaves.
- Worst-case number of compares = height h of pruned comparison tree.
- Binary tree of height h has $\leq 2^h$ leaves.

这个下界是问题本身的难度，和具体算法无关：只要你用“两两比较”的方式排序，就不可能比 $n \log n$ 更快。

$$2^h \geq \# \text{ reachable leaves} = n!$$

$$\Rightarrow h \geq \log_2(n!)$$

$$\sim n \log_2 n$$



Stirling's formula

Computational complexity

A framework to study efficiency of algorithms for solving a particular problem X .

term	description	example (X = sorting)
model of computation	<i>specifies memory and primitive operations</i>	comparison tree
cost model	<i>primitive operation counts</i>	# compares
upper bound	<i>cost guarantee provided by some algorithm for a problem</i>	$\sim n \log_2 n$
lower bound	<i>proven limit on cost guarantee for all algorithms for a problem</i>	$\sim n \log_2 n$
optimal algorithm	<i>algorithm with best possible cost guarantee for a problem</i>	mergesort

First goal of algorithm design: optimal algorithms.

Computational complexity results in context

Compares? Mergesort is **optimal** with respect to number compares.

Space? Mergesort is **not optimal** with respect to space usage.



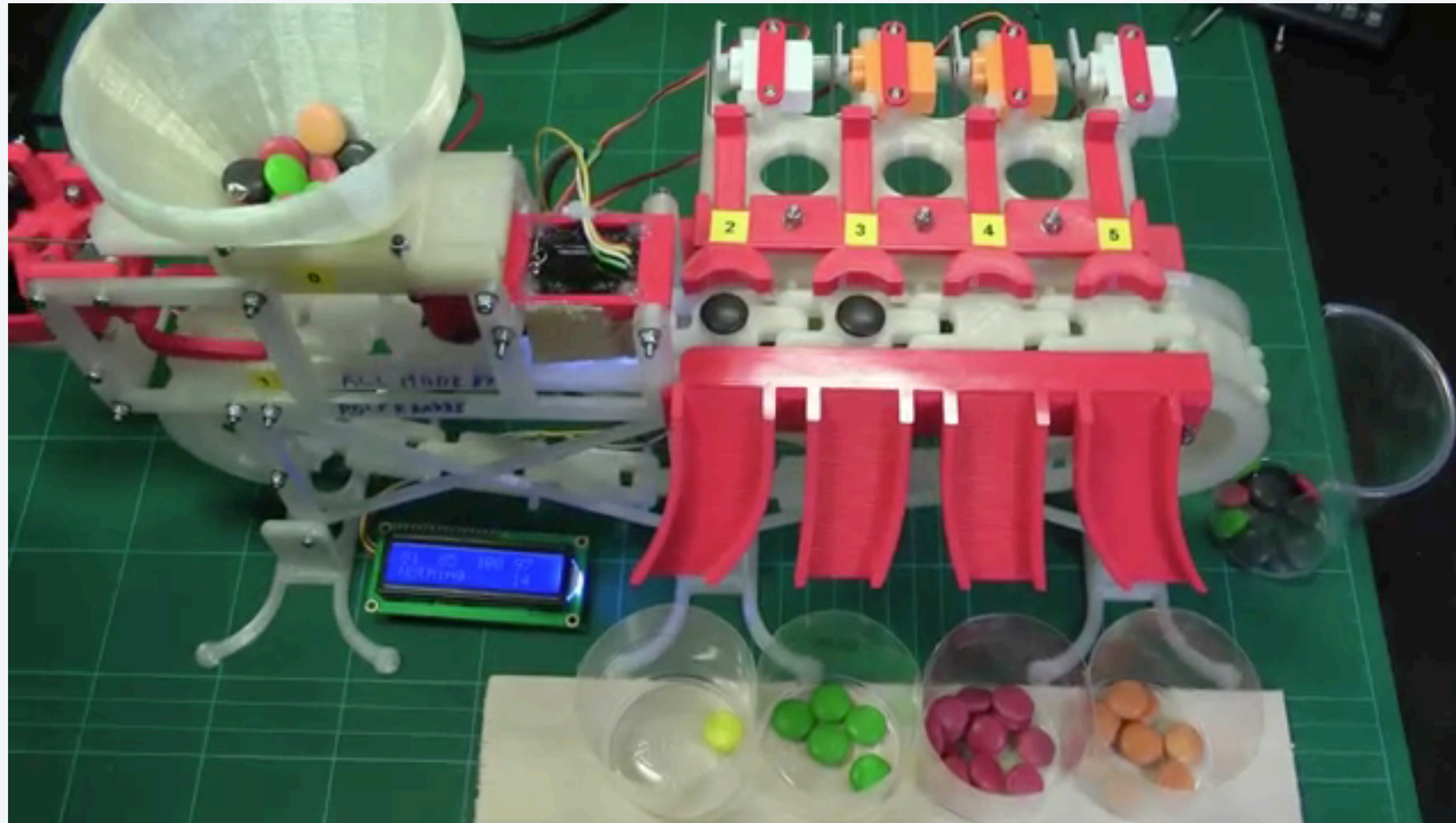
Lesson. Use theory as a guide.

Ex. ~~Design sorting algorithm that makes $\sim \frac{1}{2} n \log_2 n$ compares in worst case?~~

Ex. Design sorting algorithm that makes $\Theta(n \log n)$ compares and uses $\Theta(1)$ extra space.



Q. Why doesn't this Skittles sorter violate the sorting lower bound?



Complexity results in context (continued)

Lower bound may not hold if the algorithm can exploit:

- The initial order of the input array.

Ex: insertion sort makes only $\Theta(n)$ compares on partially sorted arrays.

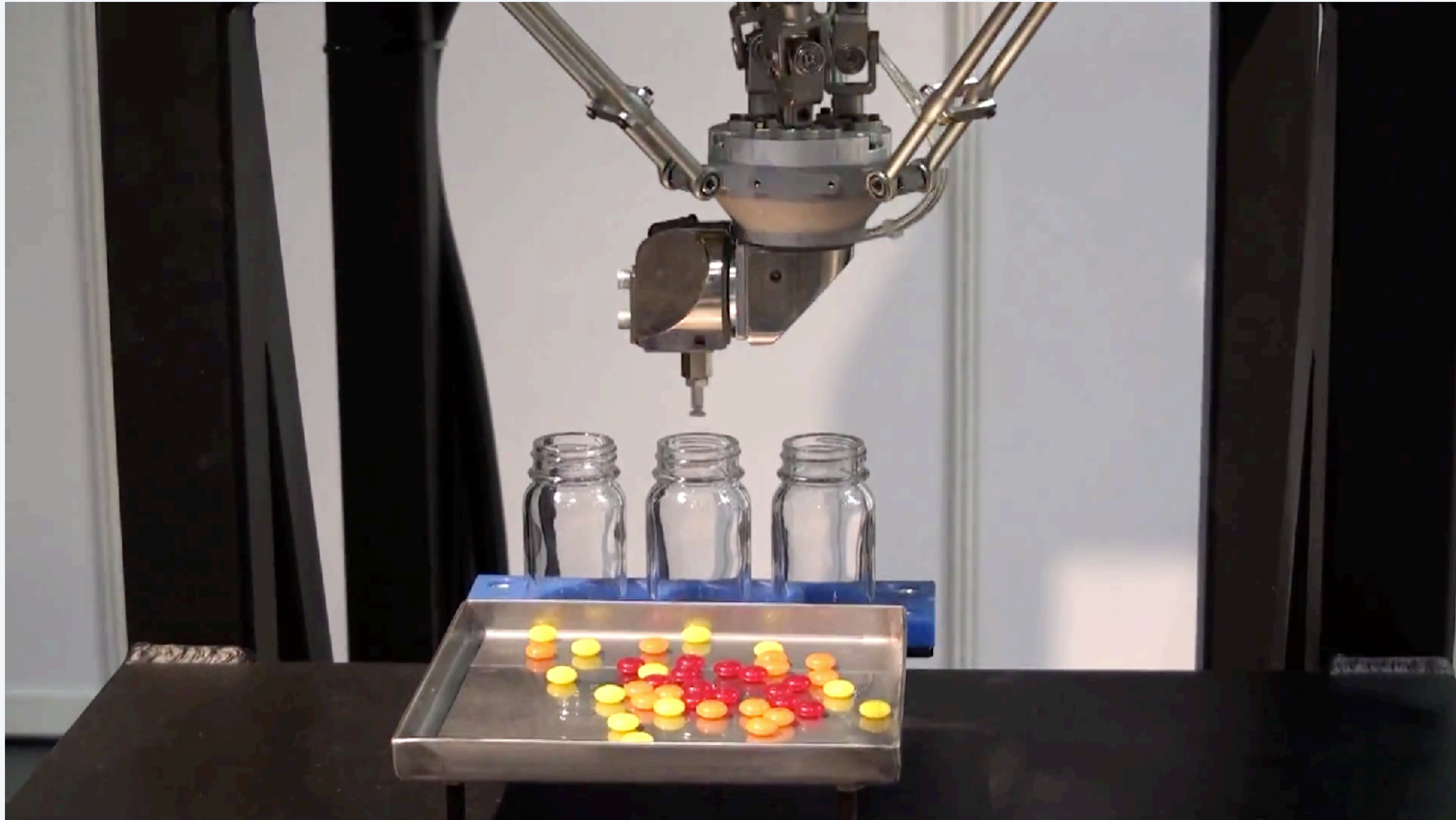
- The distribution of key values.

Ex: 3-way quicksort makes only $\Theta(n)$ compares on arrays with a small number of distinct keys. [next lecture]

- The representation of the keys.

Ex: radix sorts do not make any key compares; they access the data via individual characters/digits.

A better Skittles sorter



https://www.youtube.com/watch?v=-_JdQZTQuTI#ws

Warning: many programmers misuse O to mean Θ .

notation	provides	example	shorthand for
tilde (~)	leading term	$\sim \frac{1}{2} n^2$	$\frac{1}{2} n^2$ $\frac{1}{2} n^2 + 3 n + 22$ $\frac{1}{2} n^2 + n \log_2 n$
big Theta (Θ)	order of growth	$\Theta(n^2)$	$\frac{1}{2} n^2$ $7 n^2 + n^{\frac{1}{2}}$ $5 n^2 - 3 n$
big O (O)	upper bound	$O(n^2)$	$10 n^2$ $22 n$ $\log_2 n$
big Omega (Ω)	lower bound	$\Omega(n^2)$	$\frac{1}{2} n^2$ $n^3 + 3 n$ 2^n

ignore lower-order terms

忽略低阶项，注意完整保留高阶项！保留系数！

also ignore leading coefficient

忽略低阶项，并去掉系数

tip；先求Theta，再判断是不是O和Omega

$\Theta(n^2)$ or smaller

$\Omega \leq \Theta \leq O$

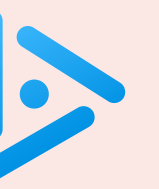
$\Theta(n^2)$ or larger

记忆：中文名越来越短



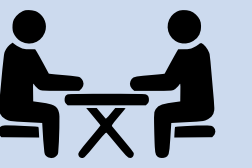
Which of the following correctly describes the function $f(n) = 3n^2 + 30n$?

- A. $\sim n^2$
- B. $\Theta(n)$
- C. $O(n^3)$
- D. *All of the above.*
- E. *None of the above.*



Which of the following correctly describes the function $f(n) = 10 \log n + 2 n \log n + 0.1 n$?

- A. $\Theta(n \log n)$
- B. $O(2^n)$
- C. $\Omega(n)$
- D. $\Omega(n \log n)$
- E. *All of **the** above.*



Interviewer. Give a formal description of the sorting lower bound for sorting arrays of n elements.

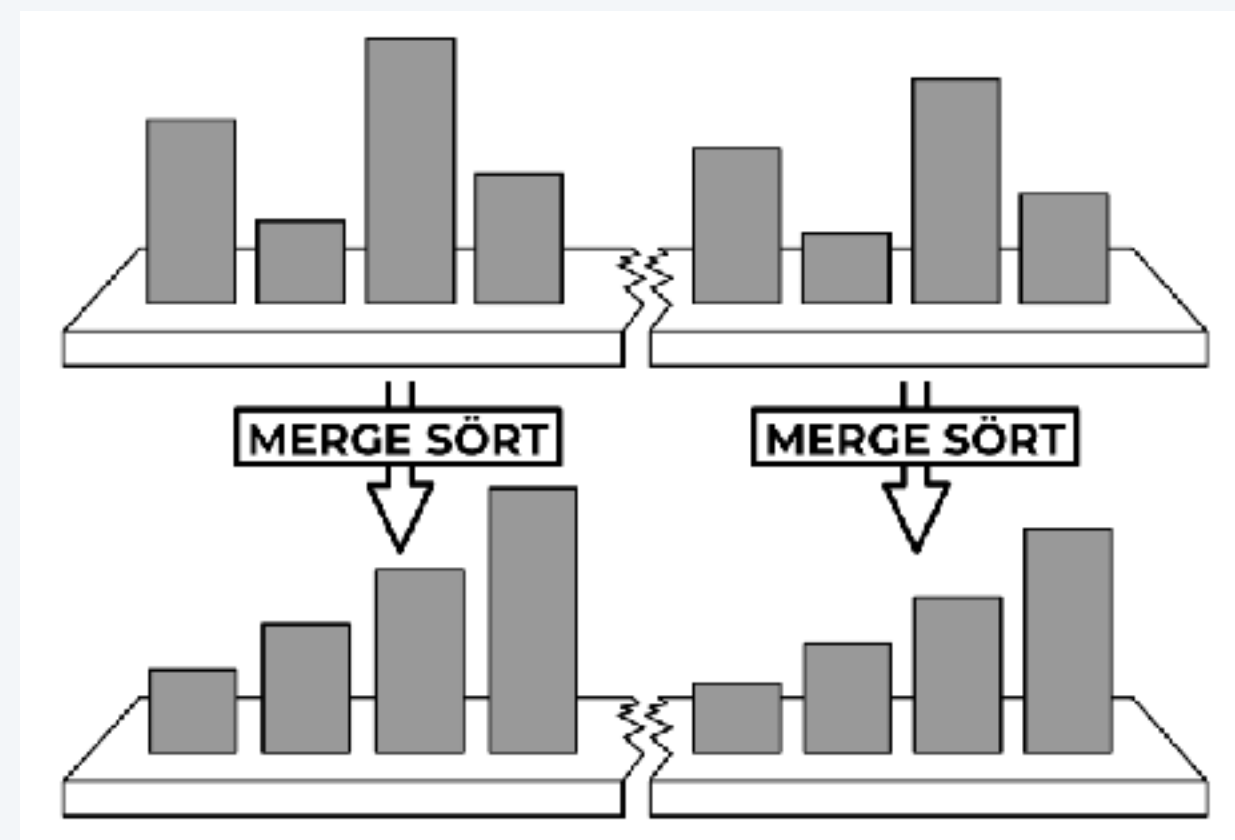


Summary

Mergesort. Makes $\Theta(n \log n)$ compares (and array accesses) in the worst case.

Sorting lower bound. No compare-based sorting algorithm makes fewer than $\Theta(n \log n)$ compares in the worst case.

Divide-and-conquer. Divide a problem into two (or more) subproblems; solve each subproblem independently; combine results.



IMPOSSIBLE



Credits

image/video	source	license
<i>Jon von Neumann</i>	<u>IAS / Alan Richards</u>	
<i>Tim Peters</i>	unknown	
<i>Theory vs. Practice</i>	<u>Ela Sjolie</u>	
<i>Skittles Sorting Machine</i>	<u>Rolf R. Bakke</u>	
<i>Fast Skittles Sorting Machine</i>	<u>Kazumichi Moriyama</u>	
<i>Impossible Stamp</i>	<u>Adobe Stock</u>	<u>education license</u>
<i>Divide-and-Conquer</i>	<u>wallpapercrafter.com</u>	
<i>Mergesort Instructions</i>	<u>IDEA</u>	<u>CC BY-NC-SA 4.0</u>

A final thought

MERGE SÖRT

idea-instructions.com/merge-sort/
v1.2, CC by-nc-sa 4.0

IDEA

